

Python & FastAPI – 2-Week Learning Overview & Practice Requirements

1. Overview of the First Two Weeks

During the first two weeks, students have completed four sessions covering **Python fundamentals, Object-Oriented Programming, and FastAPI API development.**

Week 1 – Python Fundamentals & OOP

The first week focused on building a strong foundation in Python.

Topics covered:

- Python fundamentals and syntax
- Variables and data types
- Conditional statements
- Loops
- Functions
- Lists, Tuples, Sets, and Dictionaries
- Basic Python programming logic
- Object-Oriented Programming fundamentals
- Classes and Objects

Students have already practiced the Python fundamentals during the first week and are expected to be comfortable with basic Python programming.

2. Week 2 – FastAPI & API Development

The second week focused on understanding FastAPI and building APIs using Python.

Topics covered:

- Introduction to FastAPI
- Creating a FastAPI application
- Creating API endpoints
- HTTP methods
- CRUD operations

- Create
- Read
- Update
- Delete
- Path Parameters
- Query Parameters
- Request and Response handling
- Pydantic models
- Request schemas
- Response schemas
- Designing structured API requests and responses
- Building a simple chatbot API using Groq

Students should now understand the basic flow of creating an API, receiving input, processing the request, and returning a response.

3. Week 2 – FastAPI Practice Requirements

The primary focus of this week's practice should be **FastAPI application development**.

Students should practice the following concepts independently.

Practice 1 – Basic CRUD Operations

Create a simple FastAPI application that performs CRUD operations on a collection of data.

For example, create a student management API using a Python dictionary or list.

The API should support:

```
POST /students
GET /students
GET /students/{student_id}
PUT /students/{student_id}
DELETE /students/{student_id}
```

Students should understand what each HTTP method is used for and how the API processes each request.

4. Practice 2 – Working with a Dictionary

Create a Python dictionary containing sample data.

Example:

```
students = {  
    101: {  
        "name": "John",  
        "course": "Python",  
        "age": 22  
    },  
    102: {  
        "name": "David",  
        "course": "FastAPI",  
        "age": 24  
    },  
    103: {  
        "name": "Sarah",  
        "course": "Python",  
        "age": 21  
    }  
}
```

Students should create APIs to retrieve and manipulate this data.

Expected Practice

Students should be able to:

- Retrieve all records
- Retrieve a specific record using its ID
- Add a new record
- Update an existing record
- Delete a record

5. Practice 3 – Path Parameters



Students should understand how to use a **Path Parameter** to retrieve a specific resource.

Example:

GET /students/101

The student ID **101** should be received through the path parameter and used to search the dictionary.

Students should practice questions such as:

1. How do you retrieve a student using **student_id**?
 2. What happens if the requested ID does not exist?
 3. How do you return a proper error response?
 4. How is a path parameter different from a query parameter?
-

6. Practice 4 – Query Parameters

Students should practice using **Query Parameters** to filter data.

Example:

GET /students?course=Python

The API should return only students whose course is Python.

Other examples:

GET /students?age=22

GET /students?course=FastAPI

GET /students?name=John

Students should understand:

- How query parameters work
- How to define query parameters in FastAPI
- How to filter dictionary/list data
- How to handle optional query parameters
- Difference between Path Parameters and Query Parameters

7. Practice 5 – Pydantic Request & Response Models

Students should practice creating Pydantic models for API requests and responses.

Example:

```
class StudentRequest(BaseModel):  
    name: str  
    age: int  
    course: str
```

Students should understand:

- Why Pydantic is used
- What a request schema is
- What a response schema is
- How data validation works
- How to define required fields
- How to define appropriate data types

They should be able to create their own models instead of copying an existing example.

8. Practice 6 – Build a Simple Groq Chatbot API

Students should create a simple chatbot using **FastAPI + Groq API**.

The expected flow is:

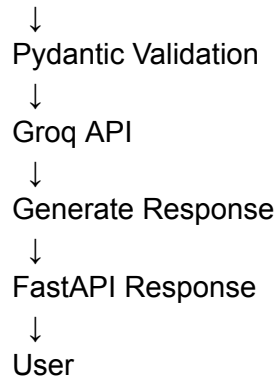
User



FastAPI Endpoint



Receive Chat Request



Students should practice creating:

POST /chat

with a request model such as:

```
class ChatRequest(BaseModel):  
    message: str
```

and return a structured response such as:

```
class ChatResponse(BaseModel):  
    response: str
```

Students should be able to explain how the request travels from the client to FastAPI, then to Groq, and finally back to the client.

9. FastAPI Concepts Students Should Explore

In addition to the concepts covered during the sessions, students are encouraged to get a basic understanding of the following FastAPI concepts before the next session.

Middleware

Understand:

- What middleware is
- Why middleware is used
- How middleware sits between the request and response
- Basic examples of middleware use cases

CORS

Understand:

- What CORS means
- Why CORS is required
- Why frontend applications may face CORS issues
- How to configure CORS in FastAPI

API Documentation

Students should also explore FastAPI's automatic API documentation:

- Swagger UI
 - ReDoc
 - How to test API endpoints using Swagger
-

10. Basic FastAPI Project Structure

Students should have a basic understanding of how a FastAPI project can be organized.

For example:

```
fastapi_project/
├── main.py
├── requirements.txt
├── .env
├── routers/
│   ├── users.py
│   └── chat.py
├── models/
│   └── schemas.py
└── services/
```

```
|   └─ chatbot.py
|
└─ utils/
    └─ helpers.py
```

Students do not need to master this structure yet.

The objective is simply to understand:

- Where the main FastAPI application is located
- Where API routes can be organized
- Where Pydantic schemas can be maintained
- Where business logic/services can be separated
- Why a project should be organized into separate modules

This will prepare students for the more structured FastAPI projects that will be introduced in the upcoming sessions.

11. Practice Checklist

Before the next session, students should be able to confidently check the following:

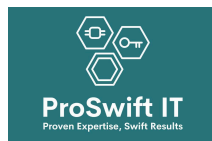
FastAPI Basics

- Create a FastAPI application from scratch
- Run the application using Uvicorn
- Create GET and POST endpoints
- Create PUT and DELETE endpoints
- Understand basic CRUD operations
- Test APIs using Swagger UI

Parameters

- Understand Path Parameters
- Use Path Parameters to retrieve a specific ID
- Understand Query Parameters
- Use Query Parameters to filter data
- Explain the difference between Path and Query Parameters

Data & Validation



- Work with dictionaries/lists as temporary data storage
- Create Pydantic models
- Create request schemas
- Create response schemas
- Understand basic request validation

Groq Integration

- Understand how an LLM API is called from FastAPI
- Create a `/chat` endpoint
- Send a user message to Groq
- Receive the model response
- Return the response through FastAPI

Additional Concepts

- Understand the purpose of Middleware
- Understand the purpose of CORS
- Explore Swagger and ReDoc
- Understand basic FastAPI project structure

12. Expected Outcome

By the end of this practice period, students should be able to **independently build a small FastAPI application** using the concepts covered during the first two weeks.

They should be able to:

**Create APIs → Perform CRUD operations → Use Path & Query Parameters →
Validate requests using Pydantic → Organize a basic FastAPI project →
Integrate Groq → Build a simple chatbot API.**

The objective is not just to memorize FastAPI syntax, but to understand **how the different components work together to build a real-world API application.**